

A1 Assigned Q2 — Retrieval Augmented Generation (RAG)

Two components: $P_r(D \mid x_{1:t})$ = retriever (probability of document D being relevant given query); $P_w(y_{1:t} \mid D, x_{1:t})$ = generator (LM producing response given document + query).

Key derivation (marginalization over D):

$$Q(y_{1:t} \mid x_{1:t}) = \sum_{D \in \mathcal{D}} P_w(y_{1:t} \mid D, x_{1:t}) \cdot P_r(D \mid x_{1:t})$$

Since the correct D is unknown, weight each document by retriever relevance and sum over all.

A1 — Q3 Notebook: Context-Aware LMs

Cross-entropy upper bound: $\ln|V|$ is the random guessing baseline (uniform predictor). A well-trained model should drive loss well below this.

LM1 (avg context) vs. LM2 (self-attention): both start near $\ln|V|$ and drop sharply early. LM2 reaches lower final loss (≈ 2.5 vs. ≈ 3.0) because self-attention assigns different weights to different past tokens, capturing richer dependencies than a simple average.

Character-level LM: learns local patterns, not semantic coherence. Low temperature = more consistent/deterministic; high temperature = more diverse but less coherent.

A1 — Q4 Notebook: BERT and Fine-tuning

BERT architecture: bidirectional attention (no masked decoding). [MASK] token used at inference to mimic masked decoding for text completion.

Selective fine-tuning: freeze all BERT layers; unfreeze only last encoder layer + classification head (MLP). LoRA with rank $l = 8$ applied via `peft` library.

Pre-trained vs. fine-tuned accuracy: pre-trained BERT ≈ 0.484 (near random) vs. fine-tuned ≈ 0.862 on IMDB sentiment. Task-specific fine-tuning is essential.

Learning rate sensitivity: too high (10^{-3}) $\rightarrow$ overfit (val loss rises above train); too low (10^{-4}) $\rightarrow$ unstable. Best at 5×10^{-4} .

A1 — Transformer / Attention (Tutorial 2)

Multi-head attention: `num_heads` = $d_{\text{embed}}/d_{\text{head}}$. Each head has its own W_q, W_k, W_v of shape $(d_{\text{embed}}, d_{\text{head}})$. Outputs concatenated then mixed through W_o of shape $(d_{\text{embed}}, d_{\text{embed}})$.

Attention computation: $Q = xW_q$, $K = xW_k$, $V = xW_v$.

$$\text{scores} = QK^\top, \quad \text{apply causal mask}, \quad \text{scale by } \frac{1}{\sqrt{d_{\text{embed}}}}, \quad \text{Softmax}, \quad \text{weighted sum of } V$$

Scaling $1/\sqrt{d_{\text{embed}}}$: prevents dot products from growing too large, which would push Softmax into saturated (near-zero gradient) regions.

Causal mask: upper-triangular matrix of $-\infty$ added to $QK^\top$ before Softmax. Future tokens become 0 after Softmax $\rightarrow$ lower-triangular attention weight matrix.

Output mixing matrix W_o : square $(d_{\text{embed}} \times d_{\text{embed}})$, applied after concatenating all head outputs. Mixes information across heads.

MLP block: two linear layers with ReLU. Intermediate size = $4 \times d_{\text{embed}}$. Applied after self-attention within each transformer block.

Transformer block structure:

$$x \rightarrow \text{SelfAttention} \rightarrow +x \rightarrow \text{RMSNorm} \rightarrow \text{MLP} \rightarrow +x \rightarrow \text{RMSNorm}$$

Skip connections pass gradients directly and prevent vanishing gradients in deep stacks.

Sinusoidal positional encoding: sin applied to even dims, cos to odd dims. Added to token embedding before transformer layers. Fixed (not learned). Injects sequential order since attention is permutation-invariant.

A2 Assigned Q1 [1.5 pts] — Discriminative & Generative Learning with Latent

Setup: Pretrained decoder $F_\theta: \mathbb{R}^m \rightarrow \mathbb{R}^d$ only (no encoder). x, z, y all discrete. Prior $P(z)$ known. F_θ deterministic $\Rightarrow P(x|z) = \mathbf{1}[F_\theta(z) = x]$. Lectures: Ch.2 (Bayes, gen vs. disc) + Ch.3 (latent space section).

2

A1 — Bigram / N-gram Models

Bigram = rank-1 Markov: $P(x_t \mid x_{1:t-1})$, only the immediately previous token matters.

Embedding matrix E is $|V| \times |V|$. Each *column* is the logit vector for that conditioning token:

$$P_E(x_t \mid x_{t-1} = j) = \text{Softmax}(E_{:,j})$$

Why always Softmax? For discrete multi-class distributions, Softmax guarantees (1) non-negativity via the exponential, and (2) outputs sum to 1 via normalization.

Log-likelihood groups by pair type. Each column of E is fully decoupled, so you can maximize over each column independently.

Training by hand: set $\partial L/\partial a = 0$. This gives:

$$\frac{e^a}{e^a + e^b} = \frac{\text{count}(A,A)}{\text{count}(A,\cdot)} = \hat{P}(A \mid A)$$

Key takeaway: MLE on a Softmax-parameterized bigram always converges to the empirical counts. The trained $P_E = \hat{P}$, NOT the true P . The gap between P_E and P is purely due to finite sampling, not a model failure.

$$P \xrightarrow{\text{finite sampling}} \hat{P} \xrightarrow{\text{MLE}} P_E = \hat{P}$$

A1 — KL Divergence (Practice Q2)

2-1: Compute $D_{\text{KL}}(P\|P_E)$ analytically using true conditional P and uniform marginal $P(x_{t-1}) = \frac{1}{2}$:

$$D_{\text{KL}}(P\|P_E) = \frac{1}{2} \sum_{x_t} P(x_t|A) \log \frac{P(x_t|A)}{P_E(x_t|A)} + \frac{1}{2} \sum_{x_t} P(x_t|B) \log \frac{P(x_t|B)}{P_E(x_t|B)}$$

where $P_E(\cdot|A) = (3/8, 5/8)$, $P_E(\cdot|B) = (4/11, 7/11)$.

2-2: $D_{\text{KL}}(\hat{P}\|P_E) = 0$, because MLE recovers $P_E = \hat{P}$ exactly. KL of a distribution from itself is always zero.

2-3: The model learns $\hat{P}$ (empirical), not the true P . They converge as corpus size $\rightarrow \infty$ by the Law of Large Numbers.

2-4: Expand the estimator by splitting the log:

$$\begin{aligned} \hat{D}_{\text{KL}}(P\|P_E) &= \underbrace{\frac{1}{B} \sum \log P(x_t|x_{t-1})}_C - \underbrace{\frac{1}{B} \sum \log P_E(x_t|x_{t-1})}_{L(E)} = C - L(E) \\ C &= \frac{1}{19} \left[3 \log \frac{3}{2} + 5 \log \frac{5}{2} + 4 \log \frac{4}{3} + 7 \log \frac{7}{3} \right] \end{aligned}$$

C is constant w.r.t. E — vanishes when optimizing. Answer left as $C - L(E)$.

2-5: Since $\hat{D}_{\text{KL}} = C - L(E)$, minimizing $\hat{D}_{\text{KL}}$ over E is identical to maximizing $L(E)$. This is the **MLE = KL minimization** equivalence.

A1 — Language Modeling (Practice Q4)

Embedding layer: 2-dim embedding $\Rightarrow E$ is a 2×9 real-valued matrix (9 = vocabulary size). Learnable parameters = $2 \times 9 = 18$.

Linear layer (logits): context is 2-dim, so `nn.Linear` maps $2 \rightarrow 9$ (one logit per vocab token).

Training samples: extract all substrings; prefix = input, next character = label. Minimize cross-entropy $\equiv$ maximize likelihood.

Parallel computation: yes. All contexts $c_t = \frac{1}{2} \sum e_i$ can be computed simultaneously across all timesteps in a batch.

A1 Assigned Q1 — Temperature

$$p = \text{Softmax}\left(\frac{z}{\theta}\right), \quad \theta > 0$$

- $\theta \rightarrow \infty$: each logit diluted ($z/\theta \rightarrow 0$) $\Rightarrow$ all entries of p equal $\Rightarrow$ **uniform distribution** (maximum randomness).
- $\theta \rightarrow 0$: logits magnified ($z/\theta \rightarrow \pm\infty$) $\Rightarrow$ max logit dominates $\Rightarrow$ **deterministic / argmax** (minimum randomness).
- θ controls randomness by scaling logits as a denominator inside Softmax. Higher θ = flatter = more creative/random. Lower θ = sharper = more conservative.

$$\text{Definition: } D_{\text{JS}}(P\|Q) = \frac{1}{2} D_{\text{KL}}(P\|M) + \frac{1}{2} D_{\text{KL}}(Q\|M), \quad M(x) = \frac{P(x) + Q(x)}{2}$$

Parts 1–3 — Properties

P1. M is a valid distribution: $M(x) \geq 0$ (average of non-negatives); $\sum_x M(x) = \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 1 = 1$.

P2. $\text{JSD} \geq 0$: Sum of two KL terms (each ≥ 0 by Gibbs' inequality) with positive weights.

P3. Symmetry: $M(x)$ is symmetric in P and Q (unchanged by swapping). Therefore:

$$D_{\text{JS}}(Q\|P) = \frac{1}{2} D_{\text{KL}}(Q\|M) + \frac{1}{2} D_{\text{KL}}(P\|M) = D_{\text{JS}}(P\|Q)$$

Part 4 — Compute KL and JSD $[P(1) = \theta, Q(1) = 0.3]$

$$\begin{aligned} M(1) &= \frac{\theta + 0.3}{2}, & M(0) &= \frac{1.7 - \theta}{2} \\ D_{\text{KL}}(P\|Q) &= \theta \log \frac{\theta}{0.3} + (1 - \theta) \log \frac{1 - \theta}{0.7} \\ D_{\text{KL}}(P\|M) &= \theta \log \frac{2\theta}{\theta + 0.3} + (1 - \theta) \log \frac{2(1 - \theta)}{1.7 - \theta} & D_{\text{KL}}(Q\|M) &= 0.3 \log \frac{0.6}{\theta + 0.3} + 0.7 \log \frac{1.4}{1.7 - \theta} \\ D_{\text{JS}}(P\|Q) &= \frac{1}{2} D_{\text{KL}}(P\|M) + \frac{1}{2} D_{\text{KL}}(Q\|M) \end{aligned}$$

Part 5 — Plot and find θ^*

Both KL and JSD minimized when $P = Q$, i.e. $\theta^* = 0.3$. Verify: $\frac{d}{d\theta} D_{\text{KL}}(P\|Q) = \log \frac{\theta}{0.3} - \log \frac{1 - \theta}{0.7} = 0 \Rightarrow \theta = 0.3$.

Part 6 — JSD = 0 iff KL = 0 (full support assumed)

$(\Rightarrow) D_{\text{JS}} = 0$: both KL terms ≥ 0 sum to zero $\Rightarrow$ each = 0 $\Rightarrow P = M$ and $Q = M \Rightarrow P = Q \Rightarrow D_{\text{KL}}(P\|Q) = 0$.
 $(\Leftarrow) D_{\text{KL}}(P\|Q) = 0 \Rightarrow P = Q \Rightarrow M = P = Q \Rightarrow$ both KL terms = 0 $\Rightarrow D_{\text{JS}} = 0$.

A2 Assigned Q3 [1 pt] — Fréchet Inception Distance (FID)

Setup: Pretrained encoder $F_\theta: x \mapsto z \sim \mathcal{N}(\mu, \Sigma)$. FID = Fréchet distance between latent distributions of real vs. generated data.

Fréchet distance (Gaussian): $D_F = \|\mu_1 - \mu_2\|^2 + \text{tr}\{\Sigma_1 + \Sigma_2 - 2(\Sigma_1 \Sigma_2)^{1/2}\}$

Tool: LLN $-\frac{1}{n} \sum_j F(x^j) \rightarrow \mathbb{E}[F(x)]$ as $n \rightarrow \infty$.

Parts 1 & 2 — Estimating $\hat{\mu}, \hat{\Sigma}$

True data: $z^j = F_\theta(x^j)$ for $x^j \sim P$.

$$\hat{\mu} = \frac{1}{n} \sum_j z^j, \quad \hat{\Sigma} = \frac{1}{n} \sum_j (z^j - \hat{\mu})(z^j - \hat{\mu})^\top$$

Generated data: sample $\hat{z}^j \sim P_w$, compute $\hat{z}^j = F_\theta(\hat{z}^j)$, same formulas $\rightarrow \hat{\mu}_w, \hat{\Sigma}_w$.

Part 3 — FID expression

$$\text{FID} = \|\hat{\mu} - \hat{\mu}_w\|^2 + \text{tr}\left\{\hat{\Sigma} + \hat{\Sigma}_w - 2(\hat{\Sigma} \hat{\Sigma}_w)^{1/2}\right\}$$

Part 4 — Encoder F_θ and n (Heusel et al., NeurIPS 2017)

4

Part 1 — Derive $Q_{w,\theta}(y \mid z)$

Marginalize $P_w(y \mid z)$ over all z consistent with x :

$$Q_{w,\theta}(y \mid x) = \frac{\sum_z P_w(y \mid z) \mathbf{1}[F_\theta(z) = x] P(z)}{\sum_{z'} \mathbf{1}[F_\theta(z') = x] P(z')}$$

Denominator = $P(x)$, independent of y — precomputable normalizer. Discrete assumption makes the sums finite.

Part 2 — Training algorithm for $P_w(y \mid z)$

Dataset gives (x^j, y^j) only — no z^j . Gradient flows only through numerator.

TrainGenLatent(D): `init w; for epochs do; for  $(x^j, y^j)$  in batch do: compute  $Q_{w,\theta}(y|x^j)$  [Part 1];  $\nabla^j \leftarrow -\nabla_w \log Q_{w,\theta}(w \leftarrow w - \eta \text{OptAvg}\{\nabla^j\})$`

Part 3 — Complexity comparison

Model	Per-sample cost	Reason
Latent $Q_{w,\theta}$	$O(K^m)$ — exponential	must enumerate all z values
Basic $P_w(y \mid x)$	$O(d)$ — linear	single forward pass
Binary latent $m = 20$:	$2^{20} \approx 10^6$ evals/sample vs. $O(d)$ for basic.	

Part 4 — Generative classifier $Q'_{w,\theta}(y \mid x)$

Given $P_w(z \mid y)$, prior $P(y)$, decoder F_θ . Express $P_w(x \mid y)$ by marginalizing over z :

$$\begin{aligned} P_w(x \mid y) &= \sum_z \mathbf{1}[F_\theta(z) = x] \cdot P_w(z \mid y) \\ Q'_{w,\theta}(y \mid x) &= \frac{\left(\sum_z \mathbf{1}[F_\theta(z) = x] P_w(z \mid y) \right) P(y)}{\sum_{y'} \left(\sum_z \mathbf{1}[F_\theta(z) = x] P_w(z \mid y') \right) P(y')} \end{aligned}$$

Key difference vs. Part 1: inner sum is y -dependent $\Rightarrow$ denominator is a genuine sum over labels (not a simple normalizer). Label prior $P(y)$ is now a required ingredient.

Part 5 — Training algorithm for $P_w(z \mid y)$

MLE: maximize $\sum_j \log P_w(x^j \mid y^j)$. No denominator sum needed — computing likelihood, not posterior. $P(y)$ not parameterized by w $\rightarrow$ no gradient through label prior.

TrainGenLatent(D): `init w; for epochs do; for  $(x^j, y^j)$  in batch do:  $P_w(x^j|y^j) = \sum_z \mathbf{1}[F_\theta(z) = x^j] \cdot P_w(z|y^j)$ ;  $\nabla^j = -\nabla_w \log P_w(x^j|y^j)$ ;  $w \leftarrow w - \eta \text{OptAvg}\{\nabla^j\}$`

Part 6 — Benefits of $Q'_{w,\theta}$ over $Q_{w,\theta}$

- Semi-supervised (strongest):** $P_w(z \mid y)$ can leverage unlabeled data via $P_w(z) = \sum_y P_w(z \mid y) P(y)$. Discriminative $P_w(y \mid z)$ cannot absorb unlabeled samples.
- Data efficiency:** Generative converges faster at small labeled dataset sizes (Ng-Jordan result). Richer class-conditional latent structure vs. fitting a decision boundary.
- Caveat:** Discriminative wins asymptotically with large labeled data.

A2 Assigned Q2 [1.5 pts] — Jensen-Shannon Divergence

1

3

QUESTION 1 (Data Distribution) We are trying to learn distribution of 3×3 images with binary pixels, i.e., $x_{i,j} \in \{0, 1\}$ for $i, j \in \{1, 2, 3\}$. Within the 1000 samples that we have collected, we have observed only two patterns: 600 samples are exactly X pattern and the remaining ones are exactly $+$ pattern, as shown below.

QUESTION 1 (Data Distribution) Data space is $\mathcal{D} = \{0, 1\}^{3 \times 3}$. There are only two matrices happening though: x^+ corresponding to pattern $+$ and x^+ corresponding to pattern $+$. The empirical distribution is then given as

$$\hat{P}(x) = \begin{cases} 0.6 & x = x^+ \\ 0.4 & x = x^- \\ 0 & x = \text{any other 510 possible matrices} \end{cases}$$

QUESTION 2 (Generative vs Discriminative Learning) We are given two pretrained generative models:

- $P_w(x)$ trained on a dataset of bird images, and
- $P_c(x)$ trained on a dataset of car images.

Assume that the images in both datasets are of the same size, and for a given x you can explicitly compute the value of $P_w(x)$ and $P_c(x)$.

Use these two models to build a *binary classifier* that classifies bird and car images. If needed, assume that the labels $y = \text{bird}$ and $y = \text{car}$ occur with the same probability, i.e.,

$$\Pr(y = \text{bird}) = \Pr(y = \text{car}) = \frac{1}{2}$$

Hint: Think of each generative model as the conditional distribution of image z given the label $y = \text{bird/car}$.

QUESTION 2 (Generative vs Discriminative Learning) We can set the conditional generative models as

$$P(x|y = \text{bird}) = P_w(x) \\ P(x|y = \text{cat}) = P_c(x)$$

We can then use Bayes' rule to write the discriminative model as

$$P(y|x) = \frac{P(x|y)}{P_w(x) + P_c(x)}$$

After basic derivation, we conclude that the classifier is

$$\text{cls}(x) = \begin{cases} \text{bird} & P_w(x) \geq P_c(x) \\ \text{cat} & \text{otherwise} \end{cases}$$

QUESTION 5 (Latent-space Generative Models) Consider data $x \in \{0, 1, 2, 3\}^d$ distributed with *unknown* data distribution $P(x)$. For this data, we have managed to find the following *deterministic* encoding-decoding pair

- encoding $F: \{0, 1, 2, 3\}^d \mapsto \{0, 1\}^m$ that maps data x to m -bit latent representation $z = F(x)$,
- decoding $\tilde{F}: \{0, 1\}^m \mapsto \{0, 1, 2, 3\}^d$ that maps the m -bit latent representation $z \in \{0, 1\}^m$ back to the data space as $\hat{x} = \tilde{F}(z)$.

We know that the encoding-decoding pair works *approximately accurate*, i.e., $x \approx \tilde{F}(F(x))$ for any data sample x . We aim to use this encoding-decoding pair to build a generative model that samples from $P(x)$.

PART I First consider the case with $d = 10$ and $m = 2$. In this case, $z = [z_1, z_2]$ with z_1 and z_2 being binary. Assume we have access to dataset $\mathcal{D}$ with 10,000 samples of $x \sim P(x)$. Passing all samples of $\mathcal{D}$ through the encoding function F , we have observed the following frequencies for latent representations:

$z_1 \backslash z_2$	0	1
0	1240	2430
1	2770	3560

The numbers in the table represent how many times each combination of $z = [z_1, z_2]$ has been seen as we pass the 10,000 samples of $\mathcal{D}$, e.g., $z = [0, 0]$ has been seen 1240 times.

- Propose a *tabular* generative model, which uses the above table along with the encoding-decoding pair to sample from data distribution. Specify how the model samples from $P(x)$.
Note: You need to propose a *tabular* model. This means that you *cannot* use any parameters as in NNs.

PART II Now, let's consider a case with $d = 10,000$ and $m = 20$. For this case, we again assume that we have access to a dataset $\mathcal{D}$ with 10,000 samples of $x \sim P(x)$.

- Explain why in this case, the basic tabular approach used in **PART I** does not work.
Hint: Pay attention to the number of samples required in this case.

- Propose a new *tabular* generative model which assumes that the latent bits are independently distributed, i.e., it assumes that the latent representation z is distributed with

$$P(z) = \prod_{i=1}^m Q_i(z_i)$$

for m Bernoulli distributions $Q_1(z_1), \dots, Q_m(z_m)$. Explain how the distributions $Q_1, \dots, Q_m$ are estimated from the dataset $\mathcal{D}$, and how we can sample from this model using the encoding-decoding pair.

QUESTION 5 (Latent-space Generative Models)

- Tabular estimate of the latent distribution is

$$\hat{P}(z) = \begin{cases} 0.124, & z = (0, 0), \\ 0.243, & z = (1, 0), \\ 0.277, & z = (0, 1), \\ 0.356, & z = (1, 1). \end{cases}$$

We can sample approximately from the data distribution by sampling $\hat{z} \sim \hat{P}(z)$ and then computing

$$\hat{x} = \tilde{F}(\hat{z})$$

- With $m = 20$, we have $2^{20} > 10^6$ cases. We cannot find a good tabular approximation of the latent distribution from only 10^4 samples.

- We first find the encoder to find $z^j = F(x^j)$ for $j = 1 : 10^4$. We approximate the distribution of each dimension independently from the samples

$$\hat{Q}_i(z_i) = \begin{cases} \frac{\# \text{ samples with } z_i^0}{10^4} & z_i = 0 \\ \frac{\# \text{ samples with } z_i^1}{10^4} & z_i = 1 \end{cases}$$

To sample a new data point, we sample $\hat{Q}_i(z_i)$ for $i = 1 : m$ independently, i.e.,

$$\hat{z} = \begin{bmatrix} \hat{z}_1 \sim \hat{Q}_1 \\ \vdots \\ \hat{z}_m \sim \hat{Q}_m \end{bmatrix}$$

We then compute the data sample as

$$\hat{x} = \tilde{F}(\hat{z})$$

This is not a necessary good idea, as there is no reason for latent variable to have independent entries.

QUESTION 3 (Autoregressive Modeling) A neural network (NN) takes as input either *three binary integers* or a special character $\emptyset$ and returns a Softmax-activated output with 8 entries. We denote this NN as

$$F_w: \{0, 1\}^3 \cup \{\emptyset\} \mapsto [0, 1]^8.$$

We aim to deploy this NN to build an autoregressive model that generates 3×3 images with binary pixels, i.e., $x_{i,j} \in \{0, 1\}$ for $i, j \in \{1, 2, 3\}$, as shown in the diagram below.

Answer the following items:

- Specify the distribution of x that is modeled by this autoregressive model in terms of F_w .
- Is this model statistically exact? Explain your answer.
Hint: Compare the modeled distribution with the data distribution $P(x)$ decomposed by the chain rule.

QUESTION 3 (Autoregressive Modeling)

- The model assumes the following approximate chain-rule

$$\hat{P}_w(x) = P(x_{1,1}, x_{2,2}, x_{3,3}) P(x_{1,2}, x_{1,3}, x_{2,3} | x_{1,1}, x_{2,2}, x_{3,3}) P(x_{2,1}, x_{3,1}, x_{3,2} | x_{1,2}, x_{1,3}, x_{2,3})$$

Using the model, we can compute this chain-rule as

- Sample $x_{1,1}, x_{2,2}, x_{3,3} \leftarrow F_w(\emptyset)$; take the corresponding Softmax entry F_w^0
- Sample $x_{1,2}, x_{1,3}, x_{2,3} \leftarrow F_w(x_{1,1}, x_{2,2}, x_{3,3})$; take the corresponding Softmax entry F_w^1
- Sample $x_{2,1}, x_{3,1}, x_{3,2} \leftarrow F_w(x_{1,2}, x_{1,3}, x_{2,3})$; take the corresponding Softmax entry F_w^2

We can compute the joint probability as

$$\hat{P}_w(x) = F_w^0 F_w^1 F_w^2$$

- No. The model does not consider the exact chain-rule. It approximates in with a reduced one.

QUESTION 4 (Energy-based Models) We aim to train an energy model $f_w: \mathbb{R}^d \mapsto \mathbb{R}$, where d is the data dimension. For this task, we are given two datasets

- $\mathcal{D} = \{x^j : j = 1, \dots, n\}$ whose samples are drawn i.i.d. from the data distribution, i.e., $x^j \sim P(x)$,
- the set $\tilde{\mathcal{D}} = \{w^j : j = 1, \dots, n\}$ with the following ideal property: for any target distribution Q , if *Langevin dynamics* is initialized from any $w^j \in \tilde{\mathcal{D}}$, it produces a sample from Q in at most 3 steps.

Explain how the set $\tilde{\mathcal{D}}$ can be leveraged to modify or improve the training loop of the energy model.

Hint: Recall the training of energy models by contrastive learning.

QUESTION 4 (Energy-based Models) In contrastive learning, we need to sample from the model. Since we know that we only need three steps if we start from a sample in $\tilde{\mathcal{D}}$, we use those samples and replace the model samples in the contrastive divergence with $\tilde{x}^j \leftarrow \tilde{\mathcal{D}}$ with w^j sampled from $\tilde{\mathcal{D}}$.

QUESTION 6 (Flow Models) A flow model $x = f_{\alpha, \beta}(z)$ maps latent $z = [z_1, z_2] \in \mathbb{R}^2$ to data $x = [x_1, x_2] \in \mathbb{R}^2$ as follows:

$$x_1 = \alpha z_1, \\ x_2 = \beta z_1 + \frac{z_2}{\alpha},$$

for learnable $\alpha, \beta \in \mathbb{R}$. You are given the following dataset with only two samples

$$\mathcal{D} = \left\{ x^{(1)} = [1, 1], x^{(2)} = [0, 0.5] \right\}.$$

We intend to train the above flow model on $\mathcal{D}$ using the maximum-likelihood method. To this end, we assume that the latent is distributed Gaussian with mean zero and variance 1, i.e., $z \sim P(z)$ where

$$P(z) = \frac{1}{2\pi} \exp \left\{ -\frac{z_1^2 + z_2^2}{2} \right\}.$$

- Specify the *inverse* flow model.
- Determine the log-likelihood on dataset $\mathcal{D}$ as a function of α and β .
Note: We define the *Jacobian matrix* as follows:

$$\nabla_z f_{\alpha, \beta}(z) = \begin{pmatrix} \frac{\partial x_1}{\partial z_1} & \frac{\partial x_1}{\partial z_2} \\ \frac{\partial x_2}{\partial z_1} & \frac{\partial x_2}{\partial z_2} \end{pmatrix}$$

Hint: The determinant of an upper-triangular matrix is given by multiplying the entries on the main diagonal.

- Find α and β , such that the likelihood determined in the previous part is maximized.

QUESTION 6 (Flow Models)

- Inverse flow is

$$z_1 = \frac{x_1}{\alpha} \\ z_2 = \alpha x_2 - \beta z_1$$

- It's easy to see that $\log \det \nabla_z f_{\alpha, \beta}(z) = 0$. Given the distribution of z , we could compute the log-likelihood on the dataset and conclude

$$\log \mathcal{L}(\alpha, \beta) \propto -\left(\frac{1}{\alpha^2} + (\alpha - \beta)^2 + \frac{\alpha^2}{4} \right)$$

- The likelihood is maximized at $\alpha = \beta = \sqrt{2}$.

QUESTION 7 (Changing Flow Direction) We want to re-do whatever we did in the lecture for flow-based models with only one simple change: we define the invertible flow model to be the one which computes the *latent representation*, i.e., we consider a flow model $f_w: \mathbb{R}^d \mapsto \mathbb{R}^d$ which for a given data sample x computes the latent representation as

$$z = f_w(x).$$

$$x = f_w^{-1}(z).$$

Considering this definition, the *inverse flow* is defined as the mapping that takes the latent representation back to the data domain, i.e.,

This formulation is used in many implementations, although it makes no computational or theoretical difference. In this exercise, we practice what we did in the lecture for this alternative representation.

- Assume that x and z are scalar, i.e., $x, z \in \mathbb{R}$, and that f_w is strictly increasing. Let $z \sim Q(z)$. Derive the data distribution $P_w(x)$ in terms of $Q(z)$ and F_w .

- Without any derivation, extend the formula derived in Part 1 to the multi-dimensional case, i.e., when $x, z \in \mathbb{R}^d$.

- Assume that we have the dataset

$$\mathcal{D} = \{x^j \text{ for } j = 1, \dots, n\}.$$

Using the maximum likelihood method, derive an empirical risk for training of the flow model on this dataset.

- Compare your derivations with what we had in the lectures. What is the key change?

QUESTION 7 (Changing Flow Direction)

- Following same approach as in the lecture notes, we end up

$$P_w(x) = Q(f_w(x)) \frac{\partial f_w}{\partial x}$$

- With same approach, we end up with

$$P_w(x) = Q(f_w(x)) |\det \nabla_x f_w(x)|$$

- The negative log-likelihood is

$$\hat{R}(w) = -\frac{1}{n} \sum_{j=1}^n [\log Q(f_w(x^j)) + \log |\det \nabla_x f_w(x^j)|]$$

- The key change is that everything is given directly in terms of flow, not its inverse. Nevertheless, as we need both networks for generation, it does not change the computation in any respect.

$F_0 = \text{Inception-v3}$, final average pooling layer $\rightarrow 2048\text{-dim } z$. $n = 50,000$ samples from each distribution (real + generated).

Key Tools (cross-cutting)
LLN: $\frac{1}{n} \sum_{j=1}^n F(x^j) \rightarrow \mathbb{E}_{x \sim P} [F(x)]$ used for: KL estimation, EBM gradient,
FID: $\frac{1}{n} \sum_{j=1}^n F(x^j) - \frac{1}{n} \sum_{j=1}^n F(y^j)$

KL properties: $D_{\text{KL}}(P||P) = 0$; $D_{\text{KL}} \geq 0$ (Gibbs); not symmetric; MLE $\equiv$ KL minimization

JSD vs KL Symmetric, bounded $[0, \log 2]$; $D_{\text{JSD}} = 0 \Leftrightarrow D_{\text{KL}} = 0$ (full support)

Temperature $\theta \rightarrow \infty$: uniform; $\theta \rightarrow 0$: argmax; scales logits as denominator in Softmax

Gen vs. Disc Gen: faster early convergence, semi-supervised enabled; Disc: wins asymptotically

MLE $\equiv$ KL $D_{\text{KL}} = C - L(E)$; minimizing KL $\equiv$ maximizing log-likelihood

Tutorial 3 — Generative vs. Discriminative Learning

- Discriminative model** directly learns $p(y|x)$ (logistic regression, SVM, neural classifiers). Only gives labels — cannot generate or sample new data.

- Generative model** learns $p(x, y) = p(x|y)p(y)$, then classifies via Bayes: $p(y|x) = \frac{p(x|y)p(y)}{\sum_y p(x|y)p(y)}$

- Extra abilities of generative models:** sample new data, estimate likelihoods, handle missing data, detect out-of-distribution inputs, support semi-supervised learning.

- Gaussian Naïve Bayes:** assumes $p(x|y) = \prod_i p(x_i|y)$ (conditional independence per feature). Learns class prior $p(y)$, per-class mean, and per-class variance. Independence assumption is often false but model is still useful.

- Ng–Jordan result (from tutorial):** on simple Gaussian data, both models achieve similar accuracy. On nonlinear data (two moons), flexible discriminative SVM learns the boundary directly; flexible generative GMM (class-conditional mixture) gets a nonlinear boundary indirectly by modeling each class density.

- Deep generative model families:**
 - Autoregressive:** $p_\theta(x) = \prod_i p_\theta(x_i | x_{<i})$ — GPT, PixelCNN, WaveNet.

- VAE:** introduces latent z ; $p_\theta(x, z) = p_\theta(x|z)p(z)$; optimizes a variational lower bound.
- GAN:** no explicit likelihood; generator $x = G_\theta(z)$ trained against a discriminator.
- Diffusion:** learns to reverse a noising process; samples by gradually denoising from noise.

Tutorial 4 — Autoregressive Models

- AR model:** predicts next value from its own past values. $\text{AR}(N)$ uses last N observations. Order N controls how much history is used.
- Stationarity assumption:** classical AR models assume statistical properties (mean, variance) do not change over time.

- Historical progression for images:**
 - PixelRNN:** adapts LM idea to images; sequential pixel prediction conditioned on all prior pixels via RNN. Powerful but slow (inherently sequential).

- PixelCNN:** replaces RNN with masked convolutions to enforce the autoregressive property while allowing **parallelized training**. Each pixel sees a fixed causal receptive field.
- PixelCNN++:** improves loss (mixture of logits instead of softmax over 256 values), more expressive conditioning, sharper outputs and more stable training.

- MaskedConv2d:** Type-A mask zeros out the current pixel and all future pixels (used for first layer). Type-B mask keeps the current pixel (used for subsequent layers). Mask is applied by multiplying weights element-wise before the convolution.
- MLP AR forecasting:** train on sliding windows of length N , predict next value. At inference, generate autoregressively — append each prediction to the context and slide forward. Larger N captures more history.

- Generation with PixelCNN:** initialize canvas to zeros, then loop over every pixel position (i, j) ; run forward pass, sample from sigmoid output at (i, j) , write back. *O(d)* serial passes required — cannot be parallelized at inference.
- GPT-2 via HuggingFace:** `AutoModelForCausalLM + AutoTokenizer; model.generate()` produces tokens autoregressively given a prompt.

Tutorial 5 — Energy-Based Models (EBMs)